

From One Node to Two

In Weeks 1 and 2 you only ever thought about a single location. A shop holding stock, or a warehouse ordering from a supplier. There was one inventory level to track and one set of costs to minimise.

Real supply chains do not work like that. A retailer does not order directly from a factory. There are intermediate layers. A regional warehouse holds bulk stock and supplies several retailers. The factory supplies the warehouse. Each layer is called an echelon, and managing inventory across multiple layers simultaneously is the core problem this project is built around.

This week we work with the simplest possible multi-echelon structure: one warehouse feeding one retailer. Two nodes. Two inventory levels. One flow of goods between them. This is small enough to draw clearly and understand completely, but complex enough that a simple formula like EOQ cannot solve it. You need PuLP.

The Structure We Are Working With



Read the diagram like this. The supplier has unlimited stock and sells to the warehouse whenever the warehouse places an order. The warehouse holds inventory and ships goods to the retailer when the retailer needs them. The retailer holds its own inventory and sells to customers. Customer demand drives everything.

Goods flow from left to right. Orders (requests for more stock) flow from right to left. When a customer buys something, the retailer eventually needs to restock from the warehouse. When the warehouse runs low, it orders from the supplier.

This week we are ignoring lead times

In real supply chains there is a delay between placing an order and receiving it. A warehouse that orders today might not receive the goods for three days.

The Inventory Balance Equation

The inventory balance equation is just accounting. It says that what you have at the end of a period equals what you had at the start, plus what arrived, minus what left.

$$\begin{aligned} \text{Inventory at end of period} &= \text{Inventory at start} + \\ &\quad \text{Received} - \text{Shipped out} \\ I[t] &= I[t-1] + \text{received}[t] - \text{shipped}[t] \end{aligned}$$

Term	What it is	Physical meaning
$I[t]$	Inventory at the end of period t	How many units are sitting in this location at the close of period t . This is what the LP solver will find for you.
$I[t-1]$	Inventory at the end of the previous period	What you had left over from last period. At $t=0$ this is your starting inventory, which you set yourself.
$\text{received}[t]$	Units that arrived this period	For the warehouse this is whatever it ordered from the supplier. For the retailer this is whatever the warehouse shipped to it.
$\text{shipped}[t]$	Units that left this period	For the warehouse this is what it sent to the retailer. For the retailer this is what customers bought, which is the demand.

Writing It for Each Node

In a two-node system you write one balance equation per node. They look slightly different because what flows into the warehouse is different from what flows into the retailer.

1

Warehouse balance equation

The warehouse receives goods from the supplier whenever it places an order. Call the warehouse order quantity o_w . The warehouse ships goods to the retailer. Call that o_r . $I_w[t] = I_w[t-1] + o_w[t] - o_r[t]$ In words: ending warehouse stock equals starting stock plus what arrived from the supplier minus what was sent to the retailer.

2

Retailer balance equation

The retailer receives goods from the warehouse. It ships goods to customers, and customer demand is a fixed known number each period. Call demand $d[t]$. $I_r[t] = I_r[t-1] + o_r[t] - d[t]$ In words: ending retailer stock equals starting stock plus what arrived from the warehouse minus what customers bought.

The o_r term appears in both equations. What the warehouse ships out is exactly what the retailer receives. This is what connects the two nodes mathematically. In PuLP both constraints share the same variable o_r , so the solver automatically ensures that whatever the warehouse sends, the retailer gets. You do not need to write a separate constraint saying they must be equal. The shared variable handles it.

The Decision Variables and Costs

Before writing the LP we need to be clear about what the solver is deciding and what we are minimising.

What the Solver Decides

Variable	Name	What it represents
o_w	Warehouse order quantity	How many units the warehouse orders from the supplier this period. This is a decision variable because we are choosing it.
o_r	Retailer replenishment	How many units the warehouse ships to the retailer this period. This is also a decision variable.
I_w	Warehouse inventory	How many units are left in the warehouse at the end of the period. The solver computes this from the balance equation.
I_r	Retailer inventory	How many units are left at the retailer at the end of the period. The solver computes this from the balance equation.

What We Are Minimising

Total cost has three parts. The objective function adds all three together.

Cost	Formula	Why we include it
Ordering cost	$c_{\text{order}} \times o_w$	Every unit ordered from the supplier costs money. We want to order only what is necessary.
Warehouse holding cost	$h_w \times I_w$	Storing goods in the warehouse costs money per unit per period.
Retailer holding cost	$h_r \times I_r$	Storing goods at the retailer also costs money, usually more than the warehouse because retail space is expensive.

So the full objective is:

Minimise: $c_{\text{order}} \times o_w$ + $h_w \times I_w$ + $h_r \times I_r$

Some Videos

Video	
Supply Chain Disruptions — Lehigh ISE 362 Channel: Larry Snyder (Lehigh University) youtube.com/watch?v=swxFx3RT_kA	Watch the first 20 minutes. Focus on the warehouse to retailer structure and why managing them separately leads to bad outcomes.

Video	
Multi Echelon Inventory Management Channel: NPTEL youtube.com/watch?v=Nrl0CtS1m8Y	This is from an IIT course and covers the same material in a more formal way. Focus on the inventory balance concept and the cost structure across echelons.

The Code : Single Period Two-Echelon LP

This is a single period model. T equals 1. There is one warehouse, one retailer, one period of demand. No lead times yet. The purpose is to get the structure right before adding complexity in Week 4.

The Complete Code

```
# WEEK 3 : Single Period Two-Echelon LP
# One warehouse, one retailer, one time period, no lead times

import pulp

# Parameters - these are the inputs we know in advance
demand = 30      # retailer must sell exactly 30 units this period
h_w     = 0.5     # holding cost at warehouse: $0.50 per unit
h_r     = 1.0     # holding cost at retailer: $1.00 per unit
c_order = 2.0     # variable cost per unit ordered from supplier
I_w0    = 50      # warehouse starts with 50 units
I_r0    = 20      # retailer starts with 20 units

# Create the LP - we are minimising total cost
m = pulp.LpProblem('TwoEchelon_SinglePeriod', pulp.LpMinimize)

# Decision variables - the solver will find these
o_w = pulp.LpVariable('o_w', lowBound=0)    # units ordered from supplier
o_r = pulp.LpVariable('o_r', lowBound=0)    # units shipped from warehouse to
retailer
I_w = pulp.LpVariable('I_w', lowBound=0)    # warehouse inventory at end of period
I_r = pulp.LpVariable('I_r', lowBound=0)    # retailer inventory at end of period

# Objective function - minimise total cost
m += c_order * o_w + h_w * I_w + h_r * I_r

# Constraint 1: Warehouse inventory balance
# Ending stock = starting stock + received from supplier - shipped to retailer
m += I_w == I_w0 + o_w - o_r, 'warehouse_balance'

# Constraint 2: Retailer inventory balance
# Ending stock = starting stock + received from warehouse - customer demand
m += I_r == I_r0 + o_r - demand, 'retailer_balance'

# Solve
m.solve(pulp.PULP_CBC_CMD(msg=0))

# Print results
print(' Solution')
print(f'Status      : {pulp.LpStatus[m.status]}')
print(f'Total cost   : ${pulp.value(m.objective):.2f}')
```

```

print()
print(f'o_w (ordered from supplier)           : {o_w.varValue:.1f} units')
print(f'o_r (shipped warehouse to retailer)   : {o_r.varValue:.1f} units')
print(f'I_w (warehouse ending inventory)      : {I_w.varValue:.1f} units')
print(f'I_r (retailer ending inventory)       : {I_r.varValue:.1f} units')

# Verify the balance equations manually
print()
print('Manual Verification ')
print(f'Warehouse check: {I_w0} + {o_w.varValue:.0f} - {o_r.varValue:.0f} = {I_w0 + o_w.varValue - o_r.varValue:.0f} (should equal I_w = {I_w.varValue:.0f})')
print(f'Retailer check : {I_r0} + {o_r.varValue:.0f} - {demand} = {I_r0 + o_r.varValue - demand:.0f} (should equal I_r = {I_r.varValue:.0f})')

```

What to Look For in the Output

When you run this code, think about the answers before accepting them. Ask yourself these questions:

- Did the solver order anything from the supplier? Why or why not? The warehouse started with 50 units, which is already more than enough to supply the retailer.
- How many units did the warehouse ship to the retailer? Is it exactly equal to the retailer's demand? Why might it be more or less?
- What is the retailer's ending inventory? If demand was 30 and the retailer started with 20, what does that tell you about how much was shipped?
- The manual verification section should print two lines where both sides of each equation are equal. If they are not equal, find the error before moving on.

Experiments

Run each experiment in order. Change one number at a time and observe what happens before moving to the next one.

Experiment 1 - What if the retailer starts with zero stock?

Change `I_r0` from 20 to 0. Run the code again. What does the solver do now? How much does the warehouse ship to the retailer? Did the total cost change? Why?

Experiment 2 - What if demand is higher than starting inventory?

Keep `I_r0` at 0 and change demand from 30 to 80. The retailer starts with nothing and needs to sell 80 units. Where does the stock come from? How does the warehouse respond? Look at `o_w` carefully.

Experiment 3 - What if holding at the retailer is very expensive?

Reset everything to the original values. Now change `h_r` from 1.0 to 5.0. Run the code. Does the ending retailer inventory change compared to the original run? The solver should try to minimise how much stock sits at the expensive location. Does it do what you would expect?

Experiment 4 - What if the warehouse has no starting stock?

Change I_{w0} to 0 and I_{r0} to 0. Demand is still 30. Run the code. What does the solver do? How much does it order from the supplier? Where does that stock go? Now change the ordering cost c_{order} from 2.0 to 0.0 and run again. Does anything change?

Experiment 5 - Try to break it

Set I_{w0} to 0, I_{r0} to 0, and add a constraint that the warehouse cannot order more than 10 units: $m_{+} = o_w \leq 10$. But keep demand at 30. What status does the solver return? Can you figure out why the problem is now infeasible? What would you need to change to make it feasible again?

Deliverable

All the understandings

All the experiments

Also try:

Extend the model to have two retailers instead of one. Retailer 1 has demand 20 and Retailer 2 has demand 15. The warehouse supplies both. You will need two new variables, o_{r1} and o_{r2} for shipments to each retailer, and two new variables I_{r1} and I_{r2} for each retailer's inventory.